

Retrievers in LangChain

Retrievers are the **core of RAG systems**.

It'll connect it with:

- Document Loaders
- Text Splitters
- Embeddings
- Vector Stores

For understanding the **full pipeline clearly**.

1. Introduction

In RAG applications, storing data is not enough.

We also need:

👉 A smart way to **find the most relevant information**

That is the job of a:

★ **Retriever**

2. Recap

So far, the RAG pipeline looks like this:

Documents → Splitter → Embeddings → Vector Store

Now we add:

User Query → Retriever → Relevant Chunks → LLM

👉 Retriever is the system that:

- Searches data
 - Finds relevant chunks
 - Sends them to the LLM
-

3. What are Retrievers?

✅ **Simple Definition:**

A **Retriever**:

Retrieves the most relevant documents/chunks for a query

💡 **Simple Analogy:**

Imagine:

- Vector Store = huge library 📖
- Retriever = librarian 🧑

You ask:

"Give me AI books"

Retriever finds the best matching books.

How Retrievers Work

Step-by-step:

1. User asks question
 2. Query converted into embedding/vector
 3. Compare with stored vectors
 4. Find similar chunks
 5. Return relevant documents
-

Example:

Query:

"What is machine learning?"

Retriever returns:

"Machine learning is a subset of AI..."

4. Types of Retrievers

There are many retrievers in LangChain.

Some common types:

- Wikipedia Retriever
 - Vector Store Retriever
 - MMR Retriever
 - Multi Query Retriever
 - Contextual Compression Retriever
-

👉 Each solves a different retrieval problem

5. Wikipedia Retriever & Code Demo

✅ What it is:

Retriever that fetches information from Wikipedia

Example Code:

```
from langchain.retrievers import WikipediaRetriever
```

```
retriever = WikipediaRetriever()
docs = retriever.get_relevant_documents("Artificial Intelligence")
```

 What happens:

- Searches Wikipedia
- Retrieves related content

 Use Case:

- General knowledge apps
- Quick external information

 Limitation:

- Depends on Wikipedia content
- Not suitable for private/custom data

6. Vector Store Retriever

 What it is:

Retriever connected to a vector store

Most Common Retriever in RAG

Example:

```
retriever = vectorstore.as_retriever()
```

 How it works:

- Query → embedding
- Search vector store
- Return similar chunks

Example Flow:

Query → Retriever → Relevant Chunks

 Use Case:

- Chat with PDF
- Document QA
- Knowledge bots

7. Maximum Marginal Relevance (MMR)

✅ What is MMR?

MMR improves retrieval by:

Returning results that are both:

- Relevant
- Diverse

🚨 Problem Without MMR

Suppose top results are:

Chunk 1 → AI definition

Chunk 2 → Same AI definition

Chunk 3 → Same AI definition

👉 Too repetitive ❌

✅ With MMR:

Retriever returns:

Chunk 1 → AI definition

Chunk 2 → Machine learning

Chunk 3 → Deep learning

💡 Key Idea:

Avoid duplicate-like results

✅ Benefit:

- Better context diversity
- More useful information

8. Multi Query Retriever

✅ What it is:

Creates **multiple versions of the user query**

💡 Why needed?

User questions may be unclear.

Example:

Original query:

"What is AI?"

Generated variations:

"Explain artificial intelligence"

"Define AI"

"What does AI mean?"

 Process:

1. Generate multiple queries
 2. Search for all
 3. Combine results
-

 Benefit:

Better retrieval coverage

 Useful When:

- Queries are ambiguous
 - User wording varies
-

9. Contextual Compression Retriever

 What it is:

Reduces retrieved content to only the important parts

 Problem:

Retriever may return long chunks.

LLMs:

- Have token limits
 - Cost money
-

 Solution:

Compress the content

Example:

Original retrieved chunk:

Long paragraph with extra details...

Compressed:

"AI is the simulation of human intelligence."

💡 Key Idea:

Keep only useful information

✅ Benefits:

- Lower token usage
 - Faster processing
 - Cleaner context
-

10. More Retrievers

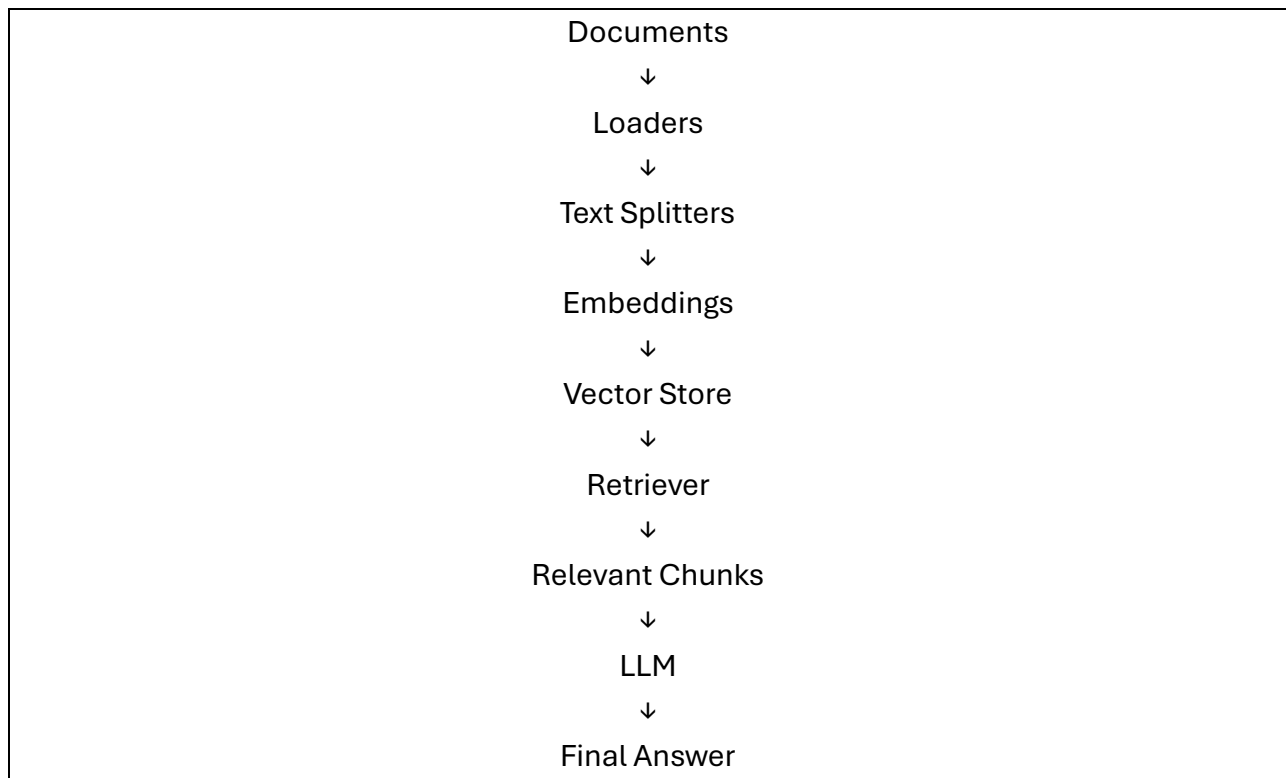
LangChain supports many retrievers:

Examples:

- SelfQueryRetriever
 - EnsembleRetriever
 - TimeWeightedRetriever
 - ParentDocumentRetriever
-

👉 Different retrievers solve different retrieval challenges

🔥 Full RAG Pipeline (VERY IMPORTANT)



Big Picture Understanding

Vector Store

Stores information

Retriever

Finds the right information

LLM

Generates final response

 Retriever is the **bridge between storage and AI reasoning**

Important Concepts to Remember

1. Retriever ≠ Database

Retriever searches intelligently.

2. Retrieval quality matters

Bad retrieval → bad answers

3. Different retrievers solve different problems

- Diversity → MMR
 - Better search coverage → MultiQuery
 - Smaller context → Compression
-

Common Mistakes

1. Retrieving too many chunks
 2. No chunk overlap
 3. Using wrong retriever type
 4. Ignoring duplicate retrievals
-

Summary

- **Retriever = finds relevant chunks for a query**
 - Core part of RAG systems
 - Types:
 - Wikipedia Retriever
 - Vector Store Retriever
 - MMR Retriever
-

- Multi Query Retriever
 - Contextual Compression Retriever
- Goal:
 - Improve relevance
 - Reduce noise
 - Give better context to LLM

👉 **Retriever = smart search engine for your AI application**